Informe Técnico — Segunda Entrega TPI Food Store (Unidades 4 a 8)

Integrantes: Tiago Guillot, Facundo Chacón y Joaquín Méndez
Materia: Bases de Datos II — Comisión 4
Proyecto integrador: Food Store
Repositorio: https://github.com/VIKROOM/DB-TPS-COM4
Commit de referencia: `fa0fc25` (scripts y evidencias, 2026-11-04) y `bac780c` (este informe)
Carácter: borrador técnico para defensa oral, elaborado exclusivamente con los archivos del repositorio.

Alcance y fuentes

Se relevaron los módulos `TP4_U4_FNBC_Desnormalizacion`, `TP5_U5_Seguridad_FoodStore`, `TP6_U6_Mantenimiento_FoodStore`, `TP7_U7_NoSQL_FoodStore` y `TP8_U8_FoodStore`, junto con los documentos raíz de trazabilidad (`DUIA*.md`, `protocolo_seguridad.md`, `informe_concurrencia.md`). Se usaron como fuente los `.md`, `.sql`, `.js`, `.ps1`, `.py`, las salidas de `evidencias/`/`evidencia/`/`logs/` y los PDF, extraídos con `pypdf` para contrastar U4, U5 y U8.

Criterio aplicado: no se completaron datos faltantes por inferencia. Cuando el repositorio no contiene un valor numérico o una salida ejecutada, se marca `[COMPLETAR DATO REAL]`; si un PDF afirma un resultado sin mostrar los números, se indica expresamente.

---

0. Checklist de cumplimiento de la consigna (U4–U8)

Matriz requisito → evidencia → estado. Las salvedades entre paréntesis remiten al faltante numerado en §6.

Requisito de la entrega	Evidencia en el repositorio / informe	Estado
U4 Refactorización de esquema heredado con redundancias validada contra DF reales y llevada a FNBC	`TP4_U4_FNBC_Desnormalizacion/` (SQL + informe); §1.1, §3.1, §4.1	Cumple
U4 Migración sin pérdida en transacciones reversibles, especificada en Kiro, con plan de reversión	`kiro/requirements.md` (REQ-01–REQ-07), `kiro/design.md` (plan de rollback), `sql/tp_fnbc_control_lote_migracion_segura.sql` (respaldo + `BEGIN/COMMIT` + verificación + `ROLLBACK`); §1.1, §2.2, §4.1	Cumple
U4 Desnormalización controlada documentada (cuándo, por qué, salvaguardas)	`v_control_lote_almacen`, `ventas_categoria_dia` y triggers; §1.1, §4.1	Cumple
U5 Esquema de usuarios, roles y permisos con mínimo privilegio	`sql/01_roles_permisos_minimo_privilegio.sql`; §1.2, §4.2	Cumple
U5 Auditoría activada y probada con simulacro de acceso indebido, reconstruido desde el log, con propuesta de contención	§2.3, §3.2; contención en `informe/Informe_TP_U5_Seguridad_FoodStore.md` §C.6	Cumple (logs son plantillas, §6.5)
U5 Auditoría de permisos asistida por IA con decisión humana documentada	`BITACORA.md`; §4.2, §5.1	Cumple
U6 Plan de mantenimiento completo (respaldo, restauración, estadísticas, limpieza, integridad) automatizado con OpenCode y documentado	`SPEC_U6.md`, `backup_foodstore.ps1`, `health_check.sql`, `maintenance.sql`, `restore_test.ps1`; §1.3, §4.3	Cumple
U6 Simulacro de desastre contra reloj con informe post-incidente	§2.4, §3.3; `informe_u6.md` («Informe post-incidente»)	Cumple
U7 Modelado documental de parte del dominio con consultas generadas y verificadas sobre datos reales	`TP7_U7_NoSQL_FoodStore/` (validador, carga, verificación, consultas); §1.4, §2.5, §3.4	Cumple

Requisito de la entrega	Evidencia en el repositorio / informe	Estado
U7 Comparación práctica documental vs. relacional y justificación de la tecnología	§3.4 (tabla comparativa) y §4.4	Cumple
U8 Base SQLite con esquema y sincronización especificados en Kiro y generados con OpenCode	`movil/schema_sqlite.sql`, `movil/semaforo.sql`, `docs/kiro/*`; §1.5, §4.5, §5.1	Cumple
U8 Sincronización simulada con conflictos probados y resolución demostrada	Casos C1–C7 y `evidencias/05_verificacion_postgres_diff.txt`; §2.6, §3.5	Cumple
U8 Code review final asistido por IA de **todos** los scripts, con decisión humana por hallazgo	`revision_codigo.py` cubre U3/U4/U5 con ejecución real (H1–H9) y U6/U7/U8 con revisión estática (H10–H17); decisión por hallazgo en `evidencias/07_revision_codigo.txt` y §3.5/§4.5	Cumple
U8 Capa de datos completa (relacional, documental, móvil) con specs, scripts en Git, mantenimiento y bitácora IA	Módulos U4–U8 versionados en commits por unidad; §7	Cumple (sin etiqueta `tp8`, §6.17)
Protocolo IA Leer, comprender, probar en copia o transacción reversible, con respaldo previo	`protocolo_seguridad.md`; §2.1, §2.2; U4 ahora usa respaldo + `BEGIN/COMMIT` + `ROLLBACK`	Cumple
Protocolo IA No enviar PII a la IA; usar datos de prueba, anonimización y enmascaramiento	`usuario_anon`, log anonimizado; §4.2, §5.1	Cumple
Protocolo IA Declarar todo uso de IA en la DUIA (herramienta, finalidad, prompts/specs)	`DUIA*.md`, `BITACORA*.md`; §5.1	Cumple (U4 sin bitácora, §6.2)
Informe Elementos implementados por unidad	§1	Cumple
Informe Cómo se probó (transacciones, copias, simulacros, conflictos)	§2	Cumple
Informe Resultados, tiempo de restauración e informe post-incidente	§3 (RTO 4,91 s, RPO 658 s) y `informe_u6.md`	Cumple
Informe Decisiones de diseño (FNBC, desnormalización, roles, documental vs. relacional)	§4	Cumple
Informe Herramientas de IA, prompts/specs, propuestas aceptadas/descartadas y justificación	§5	Cumple
Informe Capturas/resultados que verifiquen el funcionamiento	Referencias a `evidencias/`, `evidencia/`, `logs/`; §1–§3	Cumple

1. Elementos implementados por unidad (Unidades 4 a 8)

1.1 Unidad 4 — FNBC y desnormalización controlada

Módulo: `TP4_U4_FNBC_Desnormalizacion/`.

Objeto / archivo	Implementación encontrada
`sql/tp_fnbc_control_lote.sql`	Crea `control_lote_almacen(lote_id, deposito_id, responsable_control_id)` con PK `(lote_id, deposito_id)` y carga tres filas de ejemplo: `(501,30,801)`, `(502,30,801)`, `(503,31,802)`. Aplica la descomposición en `responsable_deposito` (PK `responsable_control_id`) y `control_lote` (PK `(lote_id, responsable_control_id)`, FK a `responsable_deposito`). Migra los datos con `INSERT ... ON CONFLICT DO NOTHING`. Es una demostración destructiva: usa `DROP TABLE` y recrea el ejemplo (hallazgo H7).
`sql/tp_fnbc_control_lote_migracion_segura.sql`	Versión segura de la misma migración: copia `control_lote_almacen` a `control_lote_almacen_backup`, ejecuta el DDL/DML dentro de `BEGIN ... COMMIT`, verifica ida y vuelta con `detail` y `RAISE EXCEPTION` ante filas perdidas o espurias, y deja el `ROLLBACK` documentado. Su ejecución está en `TP8_U8_FoodStore/evidencias/07_revision_codigo.txt` (H9).
Vista de compatibilidad	`v_control_lote_almacen` reconstruye `(lote_id, deposito_id, responsable_control_id)` mediante `JOIN` entre `control_lote` y `responsable_deposito`. Incluye consulta de equivalencia que debe devolver 0 filas.
`sql/tp_desnormalizacion_top_categorias.sql`	Crea `ventas_categoria_dia(fecha, categoria_id, categoria_nombre, total_vendido, cantidad_items, cantidad_pedidos, actualizado_en)` con PK `(fecha, categoria_id)` e índice `idx_vcd_fecha_total(fecha, total_vendido DESC)`.
Funciones y disparadores	`fn_recalcular_ventas_categoria_dia(date,bigint)` recalcula desde `detalle_pedido`, `producto`, `pedido` y `categoria`. `fn_trg_sync_ventas_categoria_dia()` y `fn_trg_sync_vcd_por_pedido()` sincronizan la tabla. Triggers: `trg_sync_vcd_ins`, `trg_sync_vcd_upd`, `trg_sync_vcd_del` sobre `detalle_pedido`; `trg_sync_vcd_pedido_upd` sobre `pedido`.
Carga y auditoría	Carga inicial con agregación desde la fuente de verdad y consulta de auditoría que compara `ventas_categoria_dia` contra la agregación normalizada para `CURRENT_DATE`.
Informe	`informe/Informe_TP_U4_FNBC_Desnormalizacion_FoodStore.md` documenta dependencias funcionales, clausuras, violación de FNBC, anomalías, descomposición sin pérdida, vista de compatibilidad, patrón de desnormalización y plantilla comparativa.
Spec Kiro	`kiro/requirements.md` fija REQ-01 a REQ-07 (detección de FNBC, descomposición, migración sin pérdida, transacción reversible con rollback, desnormalización, medición y uso de IA). `kiro/design.md` desarrolla el análisis FNBC, la migración transaccional, la reversión antes y después del `COMMIT`, el diseño de `ventas_categoria_dia` y el protocolo de medición.
PDF	`TP_U4_FNBC_Desnormalizacion_FoodStore.pdf` repite el análisis y afirma que la migración y la equivalencia “se verificaron”; no incluye la salida del `EXPLAIN`, los tiempos ni el resultado numérico de la auditoría.

Observaciones de consistencia: este módulo referencia `detalle_pedido`, mientras que el esquema base del repositorio (`schema.sql`) define `linea_pedido`. La correspondencia no está resuelta dentro de U4; U6 sí la resuelve con una vista de compatibilidad. El code review lo comprueba en H8: sobre el esquema final el trigger falla con `relation "detalle_pedido" does not exist`. Además, el PDF y el informe consideran `responsable_control_id` atributo primo por la clave alterna `{lote_id, responsable_control_id}`, mientras que los comentarios del SQL (`tp_fnbc_control_lote.sql:31`) lo listan como no primo. No cambia la conclusión de FNBC, pero conviene unificar la definición.

1.2 Unidad 5 — Administración y seguridad

Módulo: `TP5_U5_Seguridad_FoodStore/`.

Objeto / archivo	Implementación encontrada
`sql/01_roles_permisos_minimo_privilegio.sql`	Crea roles de grupo `rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes` y roles de login `app_web`, `admin_datos`, todos `NOINHERIT`. Asigna membresías, `USAGE` sobre `public`, permisos por objeto y permiso a nivel de columna sobre `usuario`: `SELECT(id,nombre,apellido,mail,celular,rol,eliminado,created_at)` y `UPDATE(celular)` para `rol_soporte`. Define `ALTER DEFAULT PRIVILEGES`.
`sql/02_auditoria_configuracion_y_pruebas.sql`	Documenta `log_connections=on`, `log_disconnections=on`, `log_statement='mod'` y la secuencia de pruebas: lectura, inserción, actualización e intento fallido de login.
`sql/03_simulacro_incidente_acceso_indebido.sql`	Define la secuencia del incidente controlado: intentos fallidos contra `fn_autenticar`, login exitoso, lectura masiva de `usuario` e intento de `GRANT admin_datos` a un rol de bajo privilegio. Las sentencias están comentadas como guía de ejecución.
`sql/04_usuario_anon_anonimizacion.sql`	Crea `usuario_anon` con `CREATE TABLE AS SELECT`: nombres sintéticos, mail `md5(id||salt)`, celular sintético, contraseña `***ANON***`; preserva `id`, `rol`, `eliminado`, `created_at`.
`sql/05_verificacion_equivalencia_anonimizacion.sql`	Compara conteos por `rol`, por `eliminado` y totales entre `usuario` y `usuario_anon`.
Especificación Kiro	`kiro/requirements.md` y `kiro/design.md` definen roles, justificación por rol, mínimo privilegio, auditoría, incidente, anonimización y criterios de aceptación.
Evidencias y bitácora	`logs/fragmento_log_pruebas_auditoria.txt`, `logs/log_incidente_anonimizado.txt`, `BITACORA.md` e `informe/Informe_TP_U5_Seguridad_FoodStore.md`.
PDF	`TP_U5_Seguridad_FoodStore.pdf` consolida el informe, los requisitos, el diseño y los anexos SQL/logs. Afirma que los conteos por `rol`, por `eliminado` y los totales “coinciden exactamente”, pero no muestra los valores obtenidos.

Observación de consistencia: en el repositorio no hay `CREATE TABLE usuario` ni definición de `fn_autenticar`; ambos se referencian en U5 como si existieran. El propio code review de U8 detecta el desajuste entre `usuario`/`detalle_pedido` (U5) y `cliente`/`linea_pedido` (esquema de U3).

1.3 Unidad 6 — Mantenimiento, respaldo y recuperación

Módulo: `TP6_U6_Mantenimiento_FoodStore/`.

Objeto / archivo	Implementación encontrada
`SPEC_U6.md`	Plan de respaldo diario, chequeo de salud diario, mantenimiento semanal y restauración de prueba semanal. Define RPO ≤ 24 h, RTO ≤ 20 min, reglas de seguridad y criterios de aceptación.
`backup_foodstore.ps1`	Respaldo lógico con `pg_dump -Fc -Z 6 -w`, rol `mant_backup`, snapshot previo de conteos, archivo `.parcial`, verificación con `pg_restore --list`, hash SHA-256, retención por días y registro en `mantenimiento.bitacora`.
`health_check.sql`	Controles de tamaño, conexiones/actividad, estado de tablas y autovacuum, consultas activas y bloqueos, edad de XID y antigüedad del último respaldo OK.
`maintenance.sql`	Observa autovacuum, transacciones largas y tuplas muertas; aplica `VACUUM (ANALYZE)` solo a tablas que superan el umbral (≥ 5 % y ≥ 1.000 filas, o ≥ 10.000 muertas), mediante `\gexec`; registra en `mantenimiento.bitacora`. No usa `VACUUM FULL`.

Objeto / archivo	Implementación encontrada
`restore_test.ps1`	Simulacro cronometrado sobre `foodstore_restore_test`: verifica el `.dump`, hace preflight de `rolconnlimit`, crea la base, restaura con `pg_restore --no-owner --no-privileges --exit-on-error -j 4`, valida conteos, productos inválidos, detalles inválidos y huérfanos, y escribe `restore_metrics.json`.
`soporte/`	`00_preparacion_entorno.sql` crea `mantenimiento.bitacora` y la vista de compatibilidad `public.detalle_pedido` sobre `linea_pedido`; `01_roles_mantenimiento.sql` crea `mant_backup`, `mant_monitor`, `mant_restore`; `02_simular_actividad.sql`, `conteos_validacion.sql`, `validaciones_restauracion.sql`, `setup_pgpass.ps1`, `verificar_entrega.ps1`.
Evidencias e informes	`evidencias/backup_ejecucion.log`, `backup_listado.txt`, `backup_sha256.txt`, `counts_pre_backup.txt`, `health_check_antes.txt`, `health_check_despues.txt`, `maintenance_salida.txt`, `restore_ejecucion.log`, `restore_metrics.json`, `restore_validaciones.txt`, `fallo_pg_dump.log`, `verificacion_entrega.txt`; `informe_u6.md`, `informe_u6*.pdf`, `BITACORA_IA.md`.

1.4 Unidad 7 — NoSQL y modelado documental

Módulo: `TP7_U7_NoSQL_FoodStore/`.

Objeto / archivo	Implementación encontrada
Base documental	`foodstore_nosql`, colección `pedidos`, más `meta_migracion`. Cada documento contiene `pedido_id`, `fecha`, `estado`, `forma_pago`, `cliente{cliente_id,nombre}`, `items[]` con producto, categoría, cantidad, precio unitario y subtotal, `items_count` y `total`.
`mongodb/01_crear_coleccion.js`	Crea la colección con validador `$jsonSchema`, `validationLevel:"strict"`, `validationAction:"error"`, campos obligatorios, dominios cerrados para `estado` y `forma_pago`, `cantidad>=1`, importes decimales `>=0`, `additionalProperties:false`. Crea índices: `pedido_id` único, `estado`, `items.producto_id`, `total`.
Migración	`sql/01_exportar_pedidos_mongodb.sql` genera el `insertMany()` desde PostgreSQL; `mongodb/02_datos_pedidos.js` carga 12 pedidos y 24 líneas (`pedido.id BETWEEN 1 AND 12`).
Verificación	`mongodb/03_verificacion_integridad.js` comprueba 9 condiciones: cantidad de documentos, unicidad de `pedido_id`, coherencia de `items_count`, subtotal por línea, total, dominio de estado, tipos de fecha, tipos BSON e IDs de producto válidos.
Consultas	`consultas_mongodb.js` implementa 4.a a 4.d; `consultas_postgresql.sql` implementa los equivalentes y controles de integridad; `sql/02_explain_consultas.sql` obtiene planes.
Pruebas de afirmaciones IA	`mongodb/06_pruebas_afirmaciones_ia.js` prueba el validador con `cantidad:0`, una transacción multidocumento en standalone y el plan de la consulta c).
Informes y evidencias	`informe/Informe_TP_U7_NoSQL_FoodStore.md`, `BITACORA_IA.md`, `AUTOEVALUACION.md`, `evidencia/mongo_*.txt`, `evidencia/postgres_*.txt`, `README.md`.

1.5 Unidad 8 — Esquema móvil SQLite y sincronización

Módulo: `TP8_U8_FoodStore/`.

Objeto / archivo	Implementación encontrada
`movil/schema_sqlite.sql`	Cuatro tablas `STRICT` con `PRAGMA foreign_keys=ON`: `producto_cache`, `pedido_borrador`, `item_borrador`, `cola_sync`. Precios en centavos enteros, banderas 0/1, `cantidad>0`, `solicitud_uuid` como PK de idempotencia local.
Snapshot	`servidor/00_seed_servidor.sql` crea una copia de laboratorio (`categoria`, `producto`, `cliente`, `pedido`, `linea_pedido`); `servidor/01_v_snapshot_movil.sql` define la vista con `precio_centavos=round(precio*100)`, `disponible=activo AND stock>0`, `eliminado=NOT activo`, `fecha_copia=now()`; `servidor/02_exportar_snapshot.sql` exporta a CSV.
Importación y UPSERT	`movil/importar_snapshot.sql` crea `snapshot_servidor`, normaliza booleanos `t/f` a 0/1 y refresca `producto_cache` con `INSERT ... ON CONFLICT(id) DO UPDATE`; no borra filas ausentes.
Semáforo	`movil/semaforo.sql` evalúa solo solicitudes `PENDIENTE`, agrupa por `solicitud_uuid` y clasifica con prioridad rojo > amarillo > verde: `RECHAZADO` (producto inexistente, dado de baja, no disponible o stock insuficiente), `CONFLICTO` (`PRECIO_CAMBIADO`) o `SIMULADO_APROBADO` (`TODO_COINCIDE`).
Pruebas	`movil/pruebas_sqlite.sql` prueba `COMMIT`, `ROLLBACK`, catálogo offline C1 y errores C7a (`CHECK cantidad>0`) y C7b (FK a producto inexistente). `scripts/simular_tp8.py` reproduce C2 a C6, exporta evidencias y verifica invariantes del servidor.
Code review	`scripts/revision_codigo.py` ejecuta scripts reales de U3, U4 y U5 en PostgreSQL temporal (H1–H9) y revisa estáticamente U6, U7 y U8 (H10–H17); la salida con la decisión humana por hallazgo está en `evidencias/07_revision_codigo.txt`.
Especificación y entregables	`docs/kiro/requirements.md` (REQ-01 a REQ-04), `docs/kiro/design.md`, `docs/kiro/tasks.md`, `docs/bitacora_ia.md`, `docs/DUIA.md`, `docs/informe.pdf`, `README.md` y `evidencias/`.
PDF de cierre	`docs/informe.pdf` responde las consignas 1.a, 2.c, 3.a, 3.b y 3.c: qué datos viajan y qué no, el circuito de sincronización, el mapa PostgreSQL/MongoDB/SQLite, el plan de respaldo/restauración y los hallazgos H1–H6. El PDF se generó antes de ampliar el code review a H1–H17; la salida vigente está en `evidencias/07_revision_codigo.txt`. El plan de mantenimiento es genérico y remite a U6 para las métricas reales.

2. Metodología de pruebas

2.1 Criterio general

Las pruebas del repositorio combinan tres mecanismos: ejecución sobre bases descartables, validación por consultas al motor y contraste contra salidas guardadas en `evidencias/`. La regla declarada en `protocolo_seguridad.md` es: copia de trabajo, transacción reversible (`BEGIN ... ROLLBACK/COMMIT`) y respaldo antes de cambios estructurales. No todos los módulos aplican los tres pasos con el mismo rigor; se indica en cada caso.

2.2 Transacciones reversibles

- **U8/SQLite:** `movil/pruebas_sqlite.sql` usa `BEGIN ... COMMIT` para un borrador válido, `BEGIN ... ROLLBACK` para una operación descartada y `BEGIN ... ROLLBACK` tras los errores C7a/C7b. La evidencia `evidencias/02_pruebas_sqlite.txt` muestra `1_borrador_confirmado | 1 | 2 | 1`, `2_rollback_sin_filas | 0 | 0` y `C7a/C7b | 0 | 0`.
- **U6/PostgreSQL:** el simulacro no usa una transacción única; usa una base descartable (`foodstore_restore_test`) y restaura con `--exit-on-error`. La reversibilidad se garantiza por aislamiento de la base y por la regla de no tocar `foodstore`.
- **U5/PostgreSQL:** el script de incidente propone `BEGIN; GRANT ...; ROLLBACK;` como secuencia de prueba, pero las sentencias están comentadas y no hay una salida de ejecución propia del simulacro en el repositorio.

- **U4/PostgreSQL:** la demostración `tp_fnbc_control_lote.sql` es destructiva (`DROP TABLE`) y no usa transacción; el code review la detecta como H7. La versión `tp_fnbc_control_lote_migracion_segura.sql` corrige el problema: respalda, ejecuta dentro de `BEGIN ... COMMIT` y revierte con `ROLLBACK` si la verificación falla. La prueba H9 en `TP8_U8_FoodStore/evidencias/07_revision_codigo.txt` fuerza una violación de DF2 y muestra que no quedan objetos nuevos y que la tabla original y el backup conservan sus filas.

2.3 Simulacro de acceso indebido (U5)

Secuencia documentada en `sql/03_simulacro_incidente_acceso_indebido.sql` y `kiro/design.md`:

1. Varios intentos fallidos contra `fn_autenticar`.
2. Inicio de sesión exitoso.
3. Lectura masiva de `usuario`.
4. Intento de `GRANT admin_datos` a un rol de bajo privilegio.

El log anonimizado `logs/log_incidente_anonimizado.txt` registra la secuencia y el resultado del intento de escalamiento: `ERROR: permission denied for role admin_datos`. El informe concluye que el esquema de mínimo privilegio se mantuvo. La reconstrucción por IA se contrastó críticamente: se aceptó la cronología, pero se descartaron inferencias de origen o fecha no sustentadas por el log.

Propuesta de contención (documentada en `informe/Informe_TP_U5_Seguridad_FoodStore.md` §C.6): bloquear o restringir sesiones sospechosas, revisar membresías y privilegios, forzar cambio de credenciales si corresponde, reforzar el monitoreo/auditoría y mantener el esquema de mínimo privilegio. La decisión fue del equipo, validada contra la evidencia y no automática.

Limitación de evidencia: los timestamps del log son plantillas (`[YYYY-MM-DD HH:MM:SS TZ]`), el tiempo de sesión es `<TIEMPO>` y no hay salida del `SELECT` de verificación de `information_schema.role_table_grants`.

2.4 Simulacro de desastre y restauración (U6)

Procedimiento ejecutado y evidenciado:

1. Respaldo con `pg_dump -Fc -Z 6 -w` y rol `mant_backup`.
2. Verificación del archivo con `pg_restore --list`.
3. Cálculo y registro de SHA-256.
4. Snapshot de conteos antes del respaldo.
5. Preflight de `rolconnlimit` para `pg_restore -j 4`.
6. Creación de `foodstore_restore_test` y restauración con `--no-owner --no-privileges --exit-on-error -j 4` y rol `mant_restore` (`CREATEDB`, sin superusuario).
7. Validación de conteos, `productos_invalidos`, `detalles_invalidos`, huérfanos, restricciones y vista `public.detalle_pedido`.
8. Cronómetro desde `createdb` hasta la última validación; cálculo de RTO y RPO.
9. Prueba de falla de `pg_dump` con una tabla inexistente para confirmar exit code distinto de 0 y ausencia de `.dump` válido.

2.5 Consultas y verificación cruzada en MongoDB (U7)

1. Extracción del subconjunto `pedido.id BETWEEN 1 AND 12` desde PostgreSQL.
2. Creación de `foodstore_nosql.pedidos` con validador e índices.
3. Carga de 12 documentos y 24 líneas con `insertMany()`.
4. Verificación de integridad con 9 comprobaciones ejecutadas en el motor.
5. Ejecución de consultas 4.a a 4.d en MongoDB y de las equivalentes en PostgreSQL.
6. Comparación de IDs e importes; obtención de planes con `explain` en ambos motores.
7. Prueba del validador con `cantidad:0`, de transacción multidocumento en standalone y del plan de la consulta c).

2.6 Resolución de conflictos de sincronización en SQLite (U8)

El semáforo se probó con los casos C2 a C6 sobre PostgreSQL temporal y SQLite real:

Caso	Preparación	Resultado esperado y obtenido
C1	Catálogo local sin servidor	5 productos disponibles consultados desde `producto_cache`.
C2-A	Subir el precio del producto 1 en el servidor	`CONFLICTO` / `PRECIO_CAMBIADO`.
C2-B	Aceptar el precio y enviar una solicitud nueva	`SIMULADO_APROBADO` / `TODO_COINCIDE`.
C3	Borrador por 15 unidades con stock 10	`RECHAZADO` / `STOCK_INSUFICIENTE`.
C4	Baja lógica del producto 5 después del borrador	`RECHAZADO` / `PRODUCTO_DADO_DE_BAJA`; la fila queda como lápida `(disponible=0, eliminado=1)` y el ítem del borrador se conserva.
C5	Borrador sin cambios	`SIMULADO_APROBADO` / `TODO_COINCIDE`.
C6	Reenviar el mismo UUID y repetir el semáforo	Error de PK; una sola fila y `evaluado_en` sin cambios.
C7a	Cantidad 0	`CHECK constraint failed: cantidad > 0`; 0 filas nuevas.
C7b	Producto inexistente	`FOREIGN KEY constraint failed`; 0 filas nuevas.

Además, `servidor/03_verificacion_postgres.sql` se ejecuta antes y después de toda la simulación y `evidencias/05_verificacion_postgres_diff.txt` confirma que pedidos, líneas, stock e importe no cambiaron.

---

3. Resultados obtenidos

3.1 Unidad 4

Resultado	Valor / estado
Instancia de ejemplo	3 filas: `(501,30,801)`, `(502,30,801)`, `(503,31,802)`.
Clave candidata analizada	`{lote_id, deposito_id}`; también se deriva `{lote_id, responsable_control_id}`.
¿Cumple FNBC?	No, por `DF2: responsable_control_id -> deposito_id`, cuyo determinante no es superclave.
Descomposición	`responsable_deposito(responsable_control_id, deposito_id)` y `control_lote(lote_id, responsable_control_id)`.
Equivalencia mediante vista	La consulta está definida para devolver 0 filas; no hay salida ejecutada en el módulo: `[COMPLETAR DATO REAL]`.
Demostración destructiva (H7)	Sobre una instancia con la fila `(999,99,999)`, `tp_fnbc_control_lote.sql` hace `DROP TABLE` y la fila desaparece; por eso se agregó la versión segura. Evidencia: `TP8_U8_FoodStore/evidencias/07_revision_codigo.txt`.
Prueba de rollback (H9)	Se fuerza una violación de DF2: la migración segura aborta con `RAISE EXCEPTION`, no deja `responsable_deposito`, `control_lote` ni `v_control_lote_almacen`, y conserva 2 filas en la tabla original y 2 en el backup. Evidencia: `TP8_U8_FoodStore/evidencias/07_revision_codigo.txt`.
Tiempo EXPLAIN ANALYZE antes	`[COMPLETAR DATO REAL]`.
Tiempo EXPLAIN ANALYZE después	`[COMPLETAR DATO REAL]`.
Nodo dominante antes / después	`[COMPLETAR DATO REAL]`.
Resultado de la auditoría de sincronización	La consulta debe devolver 0 filas si está sincronizado; no hay salida guardada: `[COMPLETAR DATO REAL]`.

El propio informe de U4 deja la tabla comparativa con placeholders: “(registrar valor medido)” y “Completar con las salidas reales de `EXPLAIN (ANALYZE, BUFFERS)`”. El PDF agrega que la migración “se verificó” y que la vista “coincide con la instancia original”, pero tampoco incluye la salida ni los tiempos; por eso esos resultados quedan como `[COMPLETAR DATO REAL]`.

3.2 Unidad 5

Resultado	Valor / estado
Roles creados	4 de grupo (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`) y 2 de login (`app_web`, `admin_datos`).
Permiso a nivel de columna	`SELECT` de 8 columnas de `usuario` y `UPDATE(celular)` para `rol_soporte`.
Configuración de auditoría	`log_connections=on`, `log_disconnections=on`, `log_statement='mod'`.
Intento de escalamiento	Rechazado con `ERROR: permission denied for role admin_datos` (`logs/log_incidente_anonimizado.txt:11`).
Propuesta de contención	Documentada en el informe de U5 §C.6: restringir sesiones sospechosas, revisar membresías, forzar credenciales, reforzar monitoreo y mantener mínimo privilegio.
Conteos de anonimización por rol / eliminado	No hay salida numérica guardada: `[COMPLETAR DATO REAL]`.
Total `usuario` vs `usuario_anon`	El informe y el PDF afirman que los conteos coinciden exactamente; no muestran los valores ni hay archivo de evidencia con ellos: `[COMPLETAR DATO REAL]`.
Timestamps y tiempo de sesión del incidente	Plantillas sin valor real: `[COMPLETAR DATO REAL]`.

3.3 Unidad 6

Métrica / control	Resultado registrado
Archivo de respaldo	`foodstore_20261102_224500.dump`, formato custom `-Fc`.
Tamaño	7.338.155 bytes (7,00 MB).
SHA-256	`3EFB2D089D8A14CB828684356C8CFB75A054D7AD5FDEF2D1B09CA135CB286CB3`, verificado antes de restaurar.
Entradas del TOC	45 entradas según `backup_ejecucion.log`; el encabezado de `backup_listado.txt` declara `TOC Entries: 49`. Diferencia a revisar.
RTO previsto / observado	≤ 1.200 s / **4,91 s**.
RPO previsto / observado	≤ 86.400 s / **658 s** (10 min 58 s de antigüedad del respaldo).
Conteos coincidentes	7/7 controles.
Filas restauradas	2 categorías, 20.001 clientes, 50.002 productos, 200.001 pedidos, 400.002 líneas.
Suma de cantidades	4.202.087.
Importe de líneas	11.544.138.050,00.
Productos inválidos	0.
Detalles inválidos	0.
Huérfanos referenciales	0 (pedido sin cliente, línea sin pedido, línea sin producto).
Restricciones restauradas	16.
Vista `public.detalle_pedido`	Presente, con 400.002 filas.
Autoevaluación	**102 PASS / 0 FAIL**.

Métrica / control	Resultado registrado
Problema real	Primera restauración falló con `FATAL: demasiadas conexiones para el rol "mant_restore"`; `CONNECTION LIMIT 2` no alcanzaba para `pg_restore -j 4`. Se corrigió a `CONNECTION LIMIT 10` y se repitió con éxito.
Mejora aplicada	Preflight de `rolconnlimit` en `restore_test.ps1`; para `-j 4` exige al menos 7 conexiones (5 del proceso principal y trabajadores, más margen).
Prueba de falla de `pg_dump`	Exit code 1, log `ERROR`, sin `.dump` válido (`evidencias/fallo_pg_dump.log`).
Informe post-incidente	`informe_u6.md` / `informe_u6.pdf`, con respaldo utilizado, RPO/RTO previsto vs. observado, validaciones, problema real (`CONNECTION LIMIT`) y mejora aplicada.

3.4 Unidad 7

Consulta / control	PostgreSQL	MongoDB	¿Coincide?
a) Todos los pedidos	—	12 documentos	—
b) `estado = 'TERMINADO'`	11 pedidos (`2..12`)	11 pedidos (`2..12`)	Sí
c) `items.producto_id = 1`	2 pedidos (`1,2`)	2 pedidos (`1,2`)	Sí
d) `total > 50000.00`	—	6 pedidos (`2,6,8,10,11,12`)	—
Importe b)	558.179,00	558.179,00	Sí
Importe c)	79.810,00	79.810,00	Sí
Importe d)	—	386.471,00	—
Importe total de la muestra	560.979,00	560.979,00	Sí
Documentos / líneas migradas	12 pedidos / 24 líneas / 11 clientes	12 documentos	—
Integridad	0 huérfanos, 0 duplicados de `pedido_id`	9 comprobaciones OK	Sí
Precios históricos distintos del actual	23 de 24	—	—
Plan consulta c)	*index-only scans + nested loop*, 0,086 ms	2 documentos y 2 claves examinadas, 1 ms	Resultado equivalente, no benchmark
Plan consulta b)	0,155 ms	—	—
Validador con `cantidad:0`	—	`Document failed validation`	Rechazo correcto
Transacción multidocumento	—	`Transaction numbers are only allowed on a replica set member or mongos`	Limitación del standalone

3.5 Unidad 8

Caso / control	Resultado registrado
Filas importadas en `snapshot_servidor`	5.
C2-A	`CONFLICTO` / `PRECIO_CAMBIADO` (`2026-11-05T01:54:38.841Z`).
C2-B	`SIMULADO_APROBADO` / `TODO_COINCIDE` (`2026-11-05T01:54:38.867Z`).
C3	`RECHAZADO` / `STOCK_INSUFICIENTE` (`2026-11-05T01:54:38.886Z`).
C4	`RECHAZADO` / `PRODUCTO_DADO_DE_BAJA` (`2026-11-05T01:54:39.158Z`).
C5	`SIMULADO_APROBADO` / `TODO_COINCIDE` (`2026-11-05T01:54:39.173Z`).
C6	`SIMULADO_APROBADO` / `TODO_COINCIDE` sin reevaluar (`2026-11-05T01:54:39.184Z`).
C7a	`CHECK constraint failed: cantidad > 0`; 0 filas nuevas.
C7b	`FOREIGN KEY constraint failed`; 0 filas nuevas.
Invariantes PostgreSQL	`pedido=3`, `linea_pedido=5`, `producto=5`, `cliente=3`, `suma_stock=115`, `importe_lineas=5750.00`.
Diff de invariantes	`SIN DIFERENCIAS: pedidos, lineas, stock e importe no cambiaron.`
Code review (U3/U4/U5, ejecución real)	H1–H9: H1, H2, H3, H4, H6, H7, H8 y H9 aceptados; H5 descartado porque la prueba mostró 0 huérfanos. H7 demuestra que `tp_fnbc_control_lote.sql` borra la fila 999; H8 que `detalle_pedido` no existe en el esquema final; H9 que la migración segura revierte sin dejar objetos ni perder datos.

Caso / control	Resultado registrado
Code review (U6/U7/U8, revisión estática)	H10–H17: H10, H11, H12, H13, H14, H15 y H17 aceptados como limitaciones o mejoras; H16 descartado porque `.import` y el runner Python son caminos distintos. Salida completa en `evidencias/07_revision_codigo.txt`.

4. Decisiones de diseño

4.1 Unidad 4 — FNBC y desnormalización

- **Descomponer en lugar de tolerar la violación de FNBC:** `responsable_control_id -> deposito_id` genera anomalías de inserción, borrado y actualización. La descomposición separa el dato maestro (`responsable_deposito`) de la asignación operativa (`control_lote`).
- **Unión sin pérdida:** el atributo común `responsable_control_id` es PK en `responsable_deposito` y FK en `control_lote`; cada tupla de `control_lote` se une con exactamente una de `responsable_deposito`, sin duplicar ni perder la terna original.
- **Vista de compatibilidad:** `v_control_lote_almacen` preserva la interfaz de la relación original para no romper consultas existentes.
- **Desnormalización controlada con triggers:** se eligió una columna precalculada mantenida por disparadores (`ventas_categoria_dia`) en lugar de una vista materializada o de un refresco manual. La función recalcula siempre desde la fuente de verdad (`detalle_pedido`, `producto`, `pedido`), no confía en incrementos ad-hoc. El nombre `detalle_pedido` no existe en el esquema final (`linea_pedido`); el desajuste se documenta como H8 y se propone una vista de compatibilidad.
- **Reversibilidad:** la tabla redundante puede regenerarse o eliminarse sin perder información transaccional, porque no es fuente de verdad. La migración FNBC usa `tp_fnbc_control_lote_migracion_segura.sql`: respaldo previo, `BEGIN/COMMIT`, verificación de ida y vuelta y `ROLLBACK` ante discrepancia (probado en H9).
- **Índice `(fecha, total_vendido DESC)`:** alinea el acceso del Top 5 por día con el orden solicitado.

4.2 Unidad 5 — Seguridad y mínimo privilegio

- **Roles de grupo y login separados:** permite administrar privilegios por función y asignar membresías a identidades concretas (`app_web`, `admin_datos`).
- `NOINHERIT`**:** obliga a activar explícitamente los roles cuando se requiere, reduciendo privilegios efectivos por defecto.
- **Mínimo privilegio por objeto y columna:** `rol_soporte` solo puede leer columnas necesarias de `usuario` y actualizar `celular`; no se le da `GRANT ALL`.
- **Sin** `SUPERUSER` **para** `app_web`**:** la aplicación no necesita privilegios administrativos.
- `ALTER DEFAULT PRIVILEGES`**:** busca que los objetos futuros hereden la política y no queden con permisos por defecto inseguros.
- **Auditoría nativa:** `log_connections`, `log_disconnections` y `log_statement='mod'` cubren quién, qué, cuándo y desde dónde; el detalle fila por fila requeriría `pgaudit`, que se documenta como fuera del alcance nativo.
- **Log como activo sensible:** puede contener IPs, usuarios y sentencias; se restringe su lectura a personal autorizado.
- **Anonimización determinista y sin PII:** `usuario_anon` usa `md5(id||salt)`, nombres sintéticos y contraseña `***ANON***`, preservando `id`, `rol`, `eliminado` y `created_at` para análisis agregados.
- **Rechazo de la propuesta de IA:** se descartó `GRANT ALL ON ALL TABLES` para `rol_soporte` por exceso de privilegio; se mantuvo el grant por columna.

4.3 Unidad 6 — Mantenimiento y recuperación

- **RPO/RTO explícitos:** respaldo diario para RPO ≤ 24 h y simulacro cronometrado para RTO ≤ 20 min; ambos se miden, no se estiman.
- **Roles de mínimo privilegio:** `mant_backup` (lectura + INSERT en bitácora), `mant_monitor` (`pg_monitor`), `mant_restore` (`CREATEDB` + lectura), sin superusuario.
- **Sin contraseñas en el repositorio:** autenticación vía `pgpass.conf` y uso de `-w` para fallar en vez de pedir contraseña.

- **Respaldo custom comprimido:** `-Fc -Z 6` permite restaurar selectivamente y verificar el TOC; se agrega SHA-256 y retención.
- `VACUUM (ANALYZE)` **condicionado:** se observan tuplas muertas y se actúa solo si se supera el umbral; se prohíbe `VACUUM FULL` por bloqueo y reescritura.
- **Restauración aislada:** el simulacro solo puede escribir en `foodstore_restore_test`; el script aborta si el destino es `foodstore` o no termina en `_restore_test`.
- **Preflight de conexiones:** `pg_restore -j 4` necesita el proceso principal más hasta cuatro trabajadores; se valida `rolconnlimit` antes de crear la base para evitar fallas a mitad de la restauración.
- **Compatibilidad de nombres:** se crea la vista `public.detalle_pedido` sobre `linea_pedido` para cumplir el enunciado sin alterar el DDL del modelo.
- **Un respaldo no es válido hasta restaurarlo:** la aceptación exige conteos, reglas lógicas, restricciones y vista restauradas.

4.4 Unidad 7 — Modelo documental

- **Embeber líneas:** `items[]` se lee siempre junto con el pedido, es un arreglo acotado y permite una escritura atómica por documento.
- **Referenciar cliente y producto:** se conservan `cliente_id` y `producto_id` para trazabilidad; no se copian catálogos completos.
- **Snapshot histórico:** nombre, categoría y precio unitario se congelan al momento de la venta para preservar el pedido histórico.
- `Decimal128`**:** equivale a `NUMERIC(10,2)` y evita errores de punto flotante en importes.
- **Validador** `$jsonSchema`**:** demuestra que el esquema flexible admite reglas estrictas; se usa `validationAction:"error"`.
- **Índices alineados a consultas:** `pedido_id` único, `estado`, `items.producto_id` (multikey) y `total`.
- **Estado derivado:** el esquema relacional no tiene columna de estado; se define una regla reproducible (`fecha_corte = max(fecha del lote)+1 día`; `TERMINADO` si `fecha <= corte-30 días`) aplicada igual en SQL y MongoDB.
- **Subconjunto, no migración total:** solo se migran 12 pedidos y 24 líneas, con procedencia explícita (`pedido.id BETWEEN 1 AND 12`).
- **Arquitectura combinada:** PostgreSQL sigue siendo fuente de verdad; MongoDB es una proyección de lectura. La ventaja observada es de número de operaciones, no de rendimiento demostrado.
- **Limitación de transacciones:** el `mongod` de laboratorio es standalone; no admite transacciones multidocumento. El diseño no las necesita porque pedido y líneas forman un documento.

4.5 Unidad 8 — Tablet y sincronización

- **“El móvil propone, el servidor dispone”:** la tablet solo recibe catálogo y arma borradores; no ejecuta DML sobre PostgreSQL.
- **Precio en centavos enteros:** evita diferencias de redondeo por punto flotante.
- **Baja lógica con lápida:** `eliminado=1` conserva la fila para no romper borradores viejos.
- **Idempotencia local:** `solicitud_uuid` es PK de `cola_sync`; el reenvío falla por clave primaria. El diseño aclara que la protección real debe estar en el servidor con una clave única de idempotencia y transacción.
- **Semáforo con prioridad rojo > amarillo > verde:** primero rechaza por producto inexistente, baja, no disponible o stock insuficiente; luego marca conflicto por precio; solo si todo coincide marca `SIMULADO_APROBADO`.
- **Solo se evalúan** `PENDIENTE`**:** repetir la simulación no reevalúa lo resuelto y mantiene `evaluado_en`.
- **UPSERT que conserva lápidas:** las filas que ya no vienen en la instantánea no se borran; se actualiza la lápida.
- **Datos mínimos:** no viajan clientes, usuarios, contraseñas, mails, ventas ni logs.
- `STRICT` **+ FK + CHECK:** la base local también impone integridad, no solo la aplicación.
- `SIMULADO_APROBADO` **no es una venta:** el estado es una simulación; la venta la confirma PostgreSQL.
- **Code review con decisión humana:** la ejecución real se reserva para scripts que pueden correr sobre bases descartables (U3/U4/U5); U6/U7/U8 se revisan de forma estática porque requieren servicios externos o credenciales. Cada hallazgo se acepta o descarta con justificación (H5 y H16 descartados).

---

5. Borrador de Bitácora de IA y DUIA

5.1 Tabla consolidada

Unidad	Herramienta	Finalidad / prompt documentado	Decisión registrada	Estado
U4	No hay bitácora específica en el módulo	El informe y los SQL de FNBC/desnormalización no incluyen `BITACORA` ni `DUIA` de U4.	`[COMPLETAR POR EL ALUMNO]`	Evidencia ausente
U5	OpenCode	Auditar `roles.sql` aplicando mínimo privilegio.	Propuso `GRANT ALL ON ALL TABLES` para `rol_soporte`; fue **rechazado** y se mantuvo el grant por columna.	Documentado en `BITACORA.md`
U5	Kiro	Generar `requirements` + `design` con justificación por rol.	Se **ajustaron** privilegios genéricos, se acotaron por objeto/columna y se agregó `ALTER DEFAULT PRIVILEGES`.	Documentado en `BITACORA.md`
U5	OpenCode	Reconstruir el incidente desde el log anonimizado, sin contexto extra.	Se aceptó la cronología; se **descartaron** afirmaciones no sustentadas sobre IP/origen/horario. Se confirmó `permission denied` en el `GRANT admin_datos`.	Documentado en `BITACORA.md`
U5	OpenCode	Generar script de `usuario_anon`.	Propuso copiar/truncar datos reales; fue **rechazado**. Se usaron datos sintéticos deterministas y contraseña `***ANON***`.	Documentado en `BITACORA.md`
U6	OpenCode	Generar especificación, respaldo, monitoreo, mantenimiento, restauración e informe.	Se **rechazó** `PGPASSWORD` embebido y usuario `postgres`; se **rechazó** `VACUUM FULL`; se **ajustó** el borrado automático de la base de prueba; se corrigieron `detalle_pedido`/`linea_pedido`, ruta de PostgreSQL 17, `-j` con `--single-transaction`, variables de `psql` en `-c`, `CONNECTION LIMIT` y permisos de `pgpass.conf`.	`BITACORA_IA.md` con 12 filas
U7	OpenCode	Comparar PostgreSQL y MongoDB para pedidos completos; dar ventajas, riesgos y recomendación.	Se **aceptó** el embebido de líneas; se **corrigieron** cuatro afirmaciones: transacciones multidocumento, superioridad por evitar JOIN, ausencia de integridad y crecimiento ilimitado. Se **descartó** migrar el núcleo transaccional y se adoptó arquitectura combinada.	`BITACORA_IA.md`

Unidad	Herramienta	Finalidad / prompt documentado	Decisión registrada	Estado
U8	OpenCode	Diseñar las cuatro tablas SQLite, el semáforo y el code review de U3/U5.	Se **aceptaron** `STRICT`, FK, CHECK, centavos, prioridad del semáforo y UPSERT. Se **descartó** precio `REAL`, tabla de clientes en la tablet, semáforo que solo comparaba precios y presentar `SIMULADO_APROBADO` como venta. Se **descartó H5** del code review.	`docs/bitacora_ia.md`
U8	Kiro	Requisitos EARS, diseño y tareas.	Se aceptó la estructura REQ-01 a REQ-04 y la trazabilidad requisito-caso-evidencia.	`docs/kiro/`
U8	PostgreSQL portable, SQLite, ReportLab	Verificación de vista/exportación, pruebas del semáforo y generación del PDF.	Se usó para ejecutar y evidenciar; el informe se generó desde `evidencias/`.	`docs/DUIA.md`
Global (TP1/TP2)	OpenCode, DBeaver	DUIA del TP1 y partes 1 a 3 del TP2: esquema, restricciones, concurrencia y lectura crítica.	Se aceptaron scripts tras revisión; se corrigieron casos de prueba y se descartaron scripts peligrosos sin `WHERE`/con `NOT IN`.	`DUIA.md`, `DUIA_parte1.md`, `DUIA_parte2.md`, `DUIA_parte3.md`

5.2 Espacios para completar por el alumno

Completar en la defensa oral, con evidencia concreta del repositorio:

Pregunta	Respuesta del alumno
¿Qué sugerencia de IA de U4 acepté y por qué?	`[COMPLETAR POR EL ALUMNO]`
¿Qué sugerencia de IA de U4 descarté y por qué?	`[COMPLETAR POR EL ALUMNO]`
¿Qué evidencia usé para validar la descomposición FNBC?	`[COMPLETAR POR EL ALUMNO]`
¿Qué propuesta de U5 rechacé por exceso de privilegio?	`[COMPLETAR POR EL ALUMNO]` (referencia: `GRANT ALL` para `rol_soporte`)
¿Qué inferencia del incidente descarté por no estar en el log?	`[COMPLETAR POR EL ALUMNO]`
¿Qué error de U6 detecté recién al ejecutar contra el motor?	`[COMPLETAR POR EL ALUMNO]` (referencia: `CONNECTION LIMIT 2`, variables `psql -c`, `ORDER BY` con alias)
¿Qué afirmación de IA sobre MongoDB corregí con documentación oficial?	`[COMPLETAR POR EL ALUMNO]` (referencia: transacciones, JOIN, `$jsonSchema`, límite 16 MiB)
¿Qué decisión de U8 tomé contra la sugerencia inicial de la IA?	`[COMPLETAR POR EL ALUMNO]` (referencia: precio `REAL` → centavos; clientes en tablet; semáforo solo de precios)
¿Qué hallazgo del code review descarté y por qué?	`[COMPLETAR POR EL ALUMNO]` (referencia: H5, 0 huérfanos)
¿Qué parte del informe no pude defender con evidencia del repo?	`[COMPLETAR POR EL ALUMNO]`

---

6. Faltantes detectados

Los siguientes puntos no se encontraron en el repositorio o presentan evidencia insuficiente para sostener lo afirmado en los informes:

1. **U4 — métricas de ejecución ausentes.** No hay salidas de `EXPLAIN (ANALYZE, BUFFERS)` antes/después, ni tiempos, ni resultado guardado de la consulta de equivalencia o de la auditoría de sincronización. El `.md` deja placeholders explícitos.

2. **U4 — sin bitácora/DUIA.** No hay `BITACORA` ni `DUIA` específica de la Unidad 4.
3. **U4 — desajuste de esquema (comprobado en H8).** U4 usa `detalle_pedido`; el esquema base del repositorio define `linea_pedido`. El code review ejecuta el script sobre el esquema final y captura `relation "detalle_pedido" does not exist`; no hay una vista de compatibilidad para ese nombre en U4 (la vista `v_control_lote_almacen` resuelve otro problema: la reconstrucción FNBC). Se propone crear la vista o renombrar las referencias.
4. **U5 — faltan objetos base.** No existe `CREATE TABLE usuario` ni definición de `fn_autenticar` en el repositorio; U5 los referencia.
5. **U5 — logs sin datos reales.** `logs/fragmento_log_pruebas_auditoria.txt` y `logs/log_incidente_anonimizado.txt` usan `[YYYY-MM-DD HH:MM:SS TZ]`, `[USUARIO_PRUEBA]`, `[IP_DEV]` y `<TIEMPO>`; no hay timestamps ni sesión reales.
6. **U5 — simulación no ejecutable.** `03_simulacro_incidente_acceso_indebido.sql` tiene las sentencias comentadas; no hay salida de la secuencia completa ni del `SELECT` de `information_schema.role_table_grants`.
7. **U5 — anonimización sin salida numérica.** No hay archivo con los conteos por `rol`/`eliminado` ni con el total de `usuario` vs `usuario_anon`; solo la consulta y la afirmación del informe y del PDF.
8. **U5 — defecto real de privilegios por columna.** `ALTER DEFAULT PRIVILEGES ... GRANT SELECT (columnas)` no es válido en PostgreSQL; el propio code review de U8 lo detecta como H6 y señala que el script se corta en esa línea.
9. **U6 — evidencia del primer fallo de restauración.** El informe describe `FATAL: demasiadas conexiones`, pero no hay log específico de esa primera restauración fallida; `fallo_pg_dump.log` corresponde a la prueba de falla de `pg_dump`, no a la restauración.
10. **U6 — diferencia de TOC.** `backup_ejecucion.log` informa 45 entradas y `backup_listado.txt` declara `TOC Entries: 49`; conviene aclarar si una cuenta incluye metadatos/ACL.
11. **U6 — archivos no versionados.** El `.dump` y los directorios `backups//logs/` no están en el repositorio (por diseño); no se puede reproducir la restauración sin generarlos.
12. **U7 — numeración incompleta.** No existe `mongodb/04_*.js`; las consultas 4.a a 4.d están en `consultas_mongodb.js`. No es un faltante funcional, pero sí de nomenclatura.
13. **U7 — evidencia en UTF-16.** Varios `evidencia/mongo_*.txt` están en UTF-16, lo que dificulta su lectura automática y su revisión en herramientas de texto.
14. **U7 — alcance acotado.** La muestra es de 12 pedidos; no hay benchmark ni conclusión de rendimiento generalizable.
15. **U8 — archivo referenciado inexistente.** `movil/importar_snapshot.sql` menciona una alternativa `movil/importar_snapshot.py` que no está en el repositorio.
16. **U8/U6 — plan de mantenimiento discordante.** El PDF de U8 declara 7 copias diarias y 4 semanales fuera del servidor; `backup_foodstore.ps1` usa retención configurable de 14 días y no implementa copias semanales. El PDF remite a U6 para RTO/RPO, pero no reproduce las métricas.
17. **General — sin etiqueta `tp8`.** El README de U8 menciona “el `.zip` o la etiqueta `tp8`”, pero `git tag -l` no muestra etiquetas.
18. **General — trazabilidad de IA en commits.** Los mensajes de commit no atribuyen uso de IA; la trazabilidad está solo en los `BITACORA*.md` y `DUIA*.md`.
19. **U4 — inconsistencia sobre atributos primos (aclarada).** El PDF y el informe consideran primo a `responsable_control_id` por la clave alterna `{lote_id, responsable_control_id}`, mientras que el comentario del SQL (`tp_fnbc_control_lote.sql:31`) lo lista como no primo. `kiro/design.md` unifica la definición: al formar parte de una clave candidata, es primo. No cambia la conclusión de FNBC; conviene corregir el comentario del SQL para la defensa.
20. **U4 — spec de Kiro y plan de reversión (resuelto).** Se agregaron `kiro/requirements.md` (REQ-01–REQ-07) y `kiro/design.md`, y `sql/tp_fnbc_control_lote_migracion_segura.sql` con respaldo, `BEGIN/COMMIT`, verificación y `ROLLBACK`. La prueba H9 en `evidencias/07_revision_codigo.txt` confirma que el rollback no deja objetos ni pierde datos. Queda pendiente ejecutar las métricas de `EXPLAIN` (ítem 1).
21. **U8 — alcance del code review (resuelto).** `scripts/revision_codigo.py` ahora ejecuta U3/U4/U5 (H1–H9) y revisa estáticamente U6/U7/U8 (H10–H17), con decisión humana por hallazgo. Queda como mejora opcional automatizar la ejecución de U6/U7/U8 en CI.

---

7. Conclusión operativa para defensa

El repositorio cubre las cinco unidades con implementaciones verificables: U4 demuestra la descomposición FNBC, el patrón de desnormalización y una migración segura con spec de Kiro y plan de reversión; U5 implementa roles, permisos

por columna, auditoría, incidente y anonimización; U6 tiene el paquete más fuerte de evidencia (RPO 658 s, RTO 4,91 s, 102 PASS/0 FAIL); U7 demuestra migración documental, validador, integridad y equivalencia SQL/MongoDB; U8 demuestra el esquema SQLite, el semáforo, la idempotencia y la no modificación de PostgreSQL.

Las debilidades que un examinador puede señalar son concretas: U4 no tiene métricas ejecutadas (aunque ya cuenta con spec de Kiro y plan de reversión con evidencia H9); U5 depende de objetos (`usuario`, `fn_autenticar`) que no están en el repositorio y sus logs son plantillas; U6/U7/U8 tienen faltantes menores de nomenclatura o evidencia. El code review de U8 ya cubre U3/U4/U5 con ejecución real y U6/U7/U8 con revisión estática (H1–H17). Ninguna de esas debilidades invalida los resultados respaldados por salidas reales en `evidencias/`; las métricas de U4 y los objetos base de U5 deben completarse antes de la defensa.